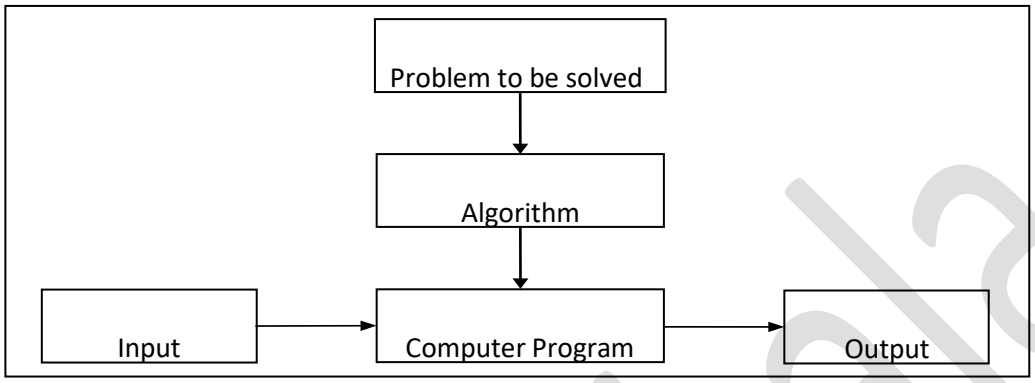
Design and Analysis of Algorithm- Unit 1

INTRODUCTION

NOTION OF AN ALGORITHM

An ***algorithm*** is a sequence of unambiguous instructions for solving a problem, i.e.,for obtaining a required output for any legitimate input in a finite amount of time.



It is a step by step procedure with the input to solve the problem in a finite amount of timeto obtain the required output.

The notion of the algorithm illustrates some important points:

- The non-ambiguity requirement for each step of an algorithm cannot be compromised.
- The range of inputs for which an algorithm works has to be specified carefully.
- The same algorithm can be represented in several different ways.
- There may exist several algorithms for solving the same problem.
- Algorithms for the same problem can be based on very different ideas and can solve the problem with dramatically different speeds.

Characteristics of an algorithm/ criteria for writing an algorithm:

Input: Zero / more quantities are externally supplied**.**

Output: At least one quantity is produced.

Definiteness: Each instruction is clear and unambiguous.

Finiteness: If the instructions of an algorithm is traced then for all cases the algorithm must terminates after a finite number of steps.

Efficiency: Every instruction must be very basic and runs in short time.

Example:

The greatest common divisor(GCD) of two nonnegative integers *m* and *n* (not-both-zero), denoted gcd*(m, n)*, is defined as the largest integer that divides both *m* and *n* evenly, i.e., with a remainder of zero.

Euclid's algorithm is based on applying repeatedly the equality gcd*(m, n)* = gcd*(n, m* mod *n),* where *m* mod *n* is the remainder of the division of *m* by *n*, until *m* mod *n* is equal to 0. Since gcd*(m,*0*)* = *m*, the last value of *m* is also the greatest common divisor of the initial *m* and *n*.

gcd*(*60*,* 24*)* can be computed as follows:gcd*(*60*,* 24*)* = gcd*(*24*,* 12*)* = gcd*(*12*,* 0*)* = 12.

SBC Karkala

Euclid's algorithm for computing gcd*(m, n)* in simple steps

Step 1 If $n = 0$, return the value of m as the answer and stop; otherwise, proceed to Step 2.

Step 2 Divide m by n and assign the value of the remainder to r.

Step 3 Assign the value of n to m and the value of r to n. Go to Step 1.

Euclid's algorithm for computing gcd*(m, n)* expressed in pseudocode

ALGORITHM *Euclid_gcd(m, n)*

```
//Computes gcd(m, n) by Euclid's algorithm
//Input: Two nonnegative, not-both-zero integers m and n
//Output: Greatest common divisor of m and n
while n ≠ 0 do
    r ←m mod n
    m←n
    n←r
return m
```

Euclid's algorithm repeatedly replaces the pair (m,n) with (n,mmodn) until n=0. The final value of m is the GCD.

Consecutive integer checking algorithm for computing gcd(m, n)

The consecutive integer checking algorithm finds the GCD by checking possible common divisors starting from the smaller of the two numbers and decreasing one by one.

Algorithm

Step 1: Assign the value of min{m, n) tot.

Step 2: Divide m by t. If the remainder of this division is 0, go to Step 3;
otherwise, go to Step 4.

Step 3: Divide n by t. If the remainder of this division is 0, return t as the answer and stop;
otherwise, proceed to Step 4.

Step 4 Decrease the value of t by 1. Go to Step 2.

Example: Find GCD of 60 and 24

Here, m=60,n=24

t=min(60,24)=24

Check 60÷24: remainder = 12 → not 0.

Decrease t to 23.

Continue checking t=23,22,…,13.

At t=12: 60÷12→remainder 0

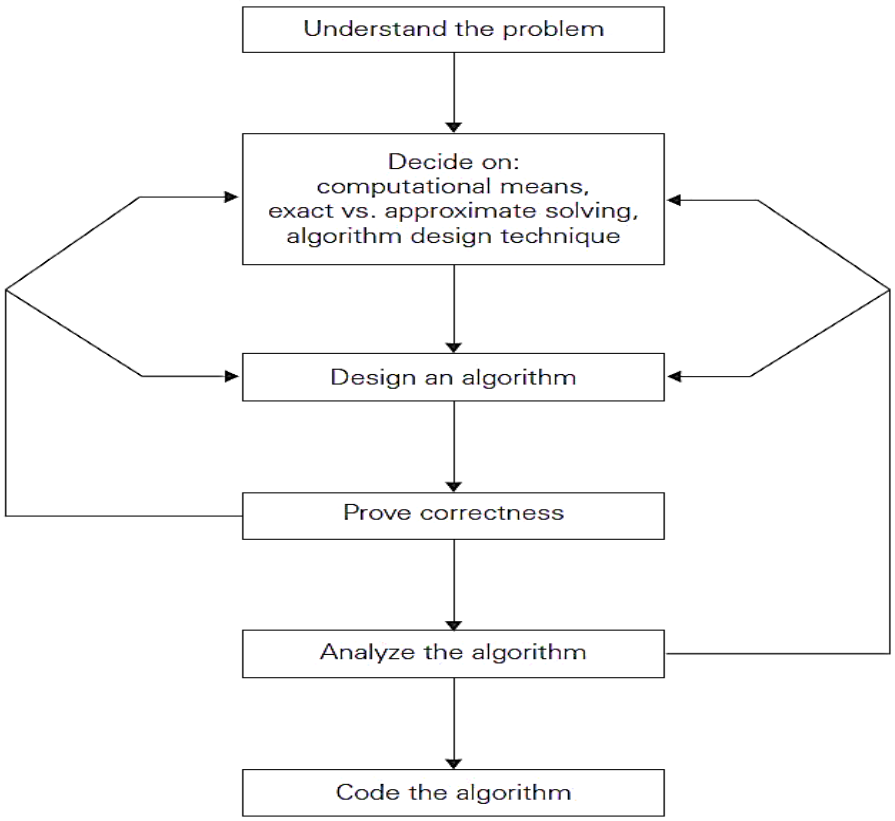
24÷12→remainder 0

Therefore, GCD(60,24)=12

SBC Karkala

FUNDAMENTALS OF ALGORITHMIC PROBLEM SOLVING

A sequence of steps involved in designing and analyzing an algorithm is shown in the figure below.



The steps of the Algorithm Design and Analysis Process are:

1. Understand and define the problem.
2. Decision making.
3. Methods of specifying an algorithm.
4. Proving an Algorithm's Correctness
5. Analyzing an Algorithm
6. Coding the algorithm.

(i) Understanding the Problem

- This is the first step in designing of algorithm.
- Read the problem's description carefully to understand the problem statement completely.
- Ask questions for clarifying the doubts about the problem.
- Identify the problem types and use existing algorithm to find solution.
- Input (***instance***) to the problem and range of the input get fixed.

(ii) Decision making

The Decision making is done on the following:

(a) Ascertaining the Capabilities of the Computational Device

- In ***random-access machine*** (***RAM***), instructions are executed one after another (The central assumption is that one operation at a time). Accordingly, algorithms designed to be executed on such machines are called ***sequential algorithms***.

- In some newer computers, operations are executed **concurrently**, i.e., in parallel. Algorithms that take advantage of this capability are called ***parallel algorithms***.
- Choice of computational devices like Processor and memory is mainly based on **space and time efficiency**

(b) Choosing between Exact and Approximate Problem Solving

- The next principal decision is to choose between solving the problem exactly or solving it approximately.
- An algorithm used to solve the problem exactly and produce correct result is calledan **exact algorithm**.

  If the problem is so complex and not able to get exact solution, then we have to choose an algorithm called an **approximation algorithm**. i.e., produces an approximate answer. Eg: extracting square roots, solving nonlinear equations, and evaluating definite integrals.

(c) Algorithm Design Techniques

- An ***algorithm design technique*** (or "strategy" or "paradigm") is a general approach to solving problems algorithmically that is applicable to a variety of problems from different areas of computing.
- **Algorithms+ *Data Structures = Programs***
- Though Algorithms and Data Structures are independent, but they are combined together to develop program. Hence the choice of proper data structure is required before designing the algorithm.
- **Implementation** of algorithm is possible only with the help of Algorithms and Data Structures
- **Algorithmic strategy / technique / paradigm** are a general approach by which many problems can be solved algorithmically. E.g., Brute Force, Divide and Conquer, Dynamic Programming, Greedy Technique and so on.

(iii) Methods of Specifying an Algorithm

There are three ways to specify an algorithm. They are**: Natural language , Pseudocode, Flowchart**

Pseudocode and flowchart are the two options that are most widely used nowadays for specifying algorithms.

a. Natural Language

It is very simple and easy to specify an algorithm using natural language. But many times specification of algorithm by using natural language is not clear and thereby we get brief specification.

Step 1: Read the first number, say a.
Step 2: Read the first number, say b.
Step 3: Add the above two numbers and store the result in c.
Step 4: Display the result from c.

Example: An algorithm to perform addition of two numbers.

Such a specification creates difficulty while actually implementing it. Hence many programmers prefer to have specification of algorithm by means of Pseudocode.

SBC Karkala

b. Pseudocode

- Pseudocode is a mixture of a natural language and programming language constructs. Pseudocode is usually more precise than natural language.
- For Assignment operation left arrow **"←"**, for comments two slashes **"//"**,**if** condition**, for, while** loops are used.

This specification is more useful for implementation of any language.

```
ALGORITHM Sum(a,b)
        //Problem Description: This algorithm performs addition of two numbers
        //Input: Two integers a and b
        //Output: Addition of two integers
        c←a+b
        return c
```

c. Flowchart

In the earlier days of computing, the dominant method for specifying algorithms was a ***flowchart***,this representation technique has proved to be inconvenient.

Flowchart is a graphical representation of an algorithm. It is a a method of expressing an algorithmby a collection of connected geometric shapes containing descriptions of the algorithm's steps.

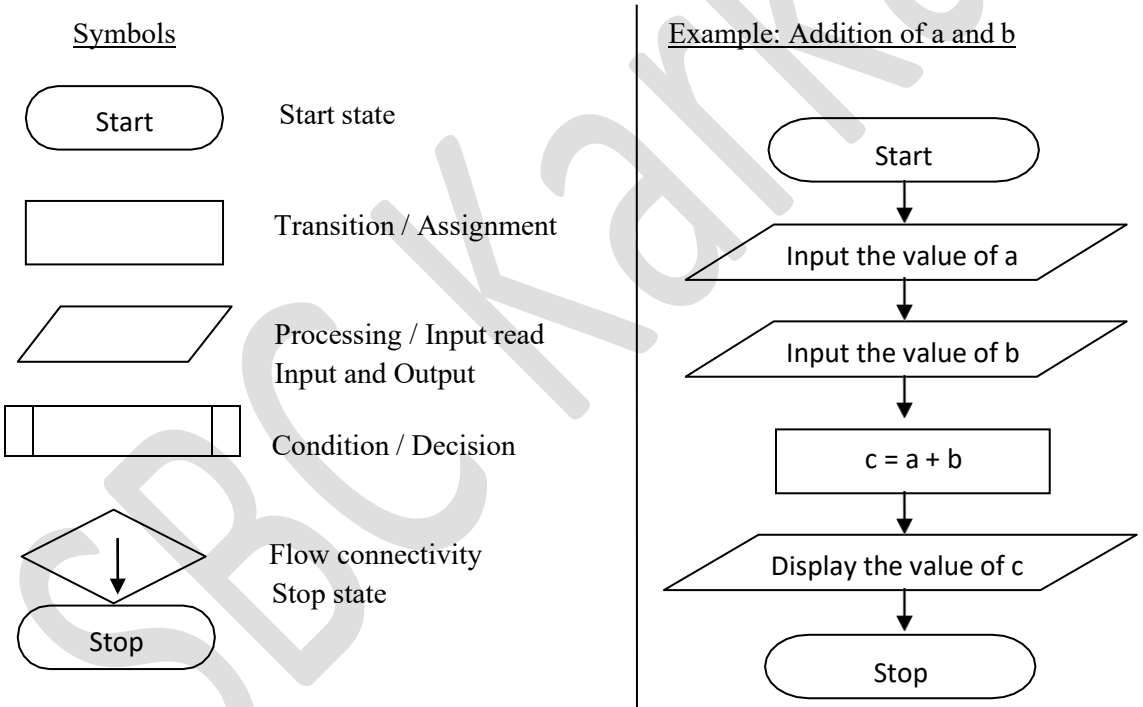


FIGURE 1.4 Flowchart symbols and Example for two integer addition.

(iv) Proving an Algorithm's Correctness

- Once an algorithm has been specified then its ***correctness*** must be proved.
- An algorithm must yields a required **result** for every legitimate input in a finite amount of time.
- For example, the correctness of Euclid's algorithm for computing the greatest common divisor stems from the correctness of the equality gcd*(m, n)* = gcd*(n, m* mod *n)*.
- A common technique for proving correctness is to use **mathematical induction** because an algorithm's iterations provide a natural sequence of steps needed for such proofs.
- The notion of correctness for approximation algorithms is less straightforward than it is for exact algorithms. The **error** produced by the algorithm should not exceed a predefined limit.

SBC Karkala

(v) Analyzing an Algorithm

- For an algorithm the most important is ***efficiency***. In fact, there are two kinds of algorithm efficiency. They are:
- ***Time Efficiency: It indicates how fast an algorithm runs or how much time it takes to complete.***
- ***Space Efficiency: It indicates how much extra memory an algorithm requires during its execution.***
- The efficiency of an algorithm is determined by measuring both time efficiency and space efficiency.
- So factors to analyze an algorithm are:
    - Time efficiency of an algorithm
    - Space efficiency of an algorithm
    - Simplicity of an algorithm
    - Generality of an algorithm

(vi) Coding an Algorithm

- An algorithm must be converted into a correct computer program.
- Every program should be tested and debugged carefully.
- User inputs should be validated in real applications.
- A program should be both correct and efficient.
- Optimizing code improves performance.
- Choosing a better algorithm gives greater speed than code optimization.
- A good algorithm is developed through repeated improvement.
- Perfection is limited by time and available resources.
- Some problems cannot be solved by any algorithm.

IMPORTANT PROBLEM TYPES

The most important problem types are:(i). Sorting
(ii). Searching
(iii). String processing
(iv). Graph problems
(v). Combinatorial problems
(vi). Geometric problems
(vii). Numerical problems

(i) Sorting

- The ***sorting problem*** is to rearrange the items of a given list in nondecreasing (ascending) order.
- Sorting can be done on numbers, characters, strings or records.
- To sort student records in alphabetical order of names or by student number or by student grade-point average. Such a specially chosen piece of information is called a ***key***.
- An algorithm is said to be **in-place** if it does not require extra memory, E.g., Quick sort.
- A sorting algorithm is called **stable** if it preserves the relative order of any two equal elements in its input.

SBC Karkala

(ii) Searching

- Searching is the algorithmic process of locating a specific target value, known as the **search key**, within a given collection or set of data items.
- It determines whether the search key exists within the data structure and, if present, returns its location or associated information.
- Searching is fundamental to databases, file systems, web search engines, and standard dictionary or contact list lookups.
- Common Examples are Linear search, binary search.

(iii) String processing

- A ***string*** is a sequence of characters from an alphabet.
- Strings comprise letters, numbers, and special characters; bit strings, which comprise zeros and ones; and gene sequences, which can be modeled by strings of characters from the four-character alphabet {A, C, G, T}. It is very useful in bioinformatics.
- Searching for a given word in a text is called string matching

(iv) Graph problems

- A graph is a collection of points called vertices, some of which are connected by line segments called edges.
- Graph problems involve finding or analyzing relationships between vertices and edges.
- Some important graph problems are graph traversal, shortest path, topological sorting, Traveling Salesman Problem (TSP), and graph coloring.
- Graph algorithms are widely used in applications such as computer networks, routing, and scheduling.

(v) Combinatorial problems

- These are problems that ask, explicitly or implicitly, to find a combinatorial object such as a permutation, a combination, or a subset that satisfies certain constraints.
- A desired combinatorial object may also be required to have some additional property suchs a maximum value or a minimum cost.
- In practical, the combinatorial problems are the most difficult problems in computing.
- The traveling salesman problem and the graph coloring problem are examples of combinatorial problems.

(vi) Geometric problems

- Geometric algorithms deal with geometric objects such as points, lines, and polygons.
- They are used to solve problems involving the position, distance, intersection, and arrangement of geometric objects.
- Geometric algorithms are used in computer graphics, robotics, and tomography.
- Important examples include the closest-pair problem and the convex-hull problem..

(vii) Numerical problems

- Numerical problems are problems that involve mathematical equations, systems of equations, definite integrals, and evaluation of functions.
- They are commonly used in scientific and engineering computations.
- The majority of numerical problems can be solved only approximately.
- Numerical methods are used to obtain approximate solutions when exact mathematical solutions are difficult or impossible to find.

SBC Karkala

FUNDAMENTALS OF THE ANALYSIS OF ALGORITHM EFFICIENCY

The efficiency of an algorithm can be in terms of time and space. The **different methods for measuring algorithm efficiency** are:

a. Analysis Framework.
b. Asymptotic Notations and its properties.
c. Mathematical analysis for Recursive algorithms.
d. Mathematical analysis for Non-recursive algorithms.

Analysis Framework

There are two kinds of efficiencies to analyze the efficiency of any algorithm. They are:

- ***Time efficiency***, indicating how fast the algorithm runs, and
- ***Space efficiency***, indicating how much extra memory it uses.

The algorithm analysis framework consists of the following:

- Measuring an Input's Size
- Units for Measuring Running Time
- Orders of Growth
- Worst-Case, Best-Case, and Average-Case Efficiencies

(i) Measuring an Input's Size

- An algorithm's efficiency is defined as a function of some parameter n indicating the algorithm's input size. In most cases, selecting such a parameter is quite straightforward. For example, it will be the size of the list for problems of sorting, searching.
- For the problem of evaluating a polynomial $p(x) = a_n x^n + \ldots + a_0$ of degree n, the size ofthe parameter will be the polynomial's degree or the number of its coefficients, which is larger by 1 than its degree.
- In computing the product of two $n \times n$ matrices, the choice of a parameter indicating an input size does matter.
- Consider a spell-checking algorithm. If the algorithm examines individual characters of its input, then the size is measured by the number of characters.
- In measuring input size for algorithms solving problems such as checking primality of a positive integer n. the input is just one number. But the input size by the number b of bits in the n's binary representation is $b=(\log_2 n)+1$.

Units for Measuring Running Time

Some standard unit of time measurement such as a second, or millisecond, and so on can beused to measure the running time of a program after implementing the algorithm.

Drawbacks

- Dependence on the speed of a particular computer.
- Dependence on the quality of a program implementing the algorithm.
- The compiler used in generating the machine code.
- The difficulty of clocking the actual running time of the program.

So, we need metric to measure an *algorithm*'s efficiency that does not depend on these extraneous factors.

SBC Karkala

One possible approach is to ***count the number of times each of the algorithm's operations is executed***. This approach is excessively difficult.

The most important operation (+, -, *, /) of the algorithm, called the ***basic operation***. Computing the number of times the basic operation is executed is easy. The total running time is determined by basic operations count.

(ii) Orders of Growth

- A difference in running times on small inputs is not what really distinguishes efficient algorithms from inefficient ones.
- For example, the greatest common divisor of two small numbers, it is not immediately clear how much more efficient Euclid's algorithm is compared to the other algorithms, thedifference in algorithm efficiencies becomes clear for larger numbers only.
- For large values of *n*, it is the function's order of growth that counts just like the Table 1.1, which contains values of a few functions particularly important for analysis of algorithms.

Orders of growth are necessary because:

1. They help compare the efficiency of algorithms for large input sizes.
2. They show how the running time increases as the input size n increases.
3. Small differences in running time may not matter for small inputs, but the difference becomes significant for large inputs.
4. They help identify which algorithm is more efficient and scalable.

TABLE 1.1 Values (approximate) of several functions important for analysis of algorithms

n	$\sqrt{n}$	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n	$n!$
1	**1**	**0**	**1**	**0**	**1**	**1**	**2**	**1**
2	**1.4**	**1**	**2**	**2**	**4**	**4**	**4**	**2**
4	**2**	**2**	**4**	**8**	**16**	**64**	**16**	**24**
8	**2.8**	**3**	**8**	**$2.4 \cdot 10^1$**	**64**	**$5.1 \cdot 10^2$**	**$2.6 \cdot 10^2$**	**$4.0 \cdot 10^4$**
10	**3.2**	**3.3**	**10**	**$3.3 \cdot 10^1$**	**10^2**	**10^3**	**10^3**	**$3.6 \cdot 10^6$**
16	**4**	**4**	**16**	**$6.4 \cdot 10^1$**	**$2.6 \cdot 10^2$**	**$4.1 \cdot 10^3$**	**$6.5 \cdot 10^4$**	**$2.1 \cdot 10^{13}$**
10^2	**10**	**6.6**	**10^2**	**$6.6 \cdot 10^2$**	**10^4**	**10^6**	**$1.3 \cdot 10^{30}$**	**$9.3 \cdot 10^{157}$**
10^3	31	10	10^3	$1.0 \cdot 10^4$	10^6	10^9	Very big computation	
10^4	10^2	13	10^4	$1.3 \cdot 10^5$	10^8	10^{12}		
10^5	$3.2 \cdot 10^2$	17	10^5	$1.7 \cdot 10^6$	10^{10}	10^{15}		
10^6	10^3	20	10^6	$2.0 \cdot 10^7$	10^{12}	10^{18}		

Sri Karkala

Types of efficiencies

Worst-Case, Best-Case, and Average-Case Efficiencies

Consider Sequential Search algorithm some search key *K*

ALGORITHM *SequentialSearch(A*[0..*n* - 1]*, K)*

```
//Searches for a given value in a given array by sequential search
//Input: An array A[0..n - 1] and a search key K
//Output: The index of the first element in A that matches K or -1 if there are no
//          matching elements
i ←0
while i < n and A[i] ≠ K do
        i ←i + 1if i
< n  return  i else
return -1
```

Clearly, the running time of this algorithm can be quite different for the same list size *n*.

In the worst case, there is no matching of elements or the first matching element can foundat last on the list. In the best case, there is matching of elements at first on the list.

Worst-case efficiency

- The worst-case efficiency of an algorithm is its efficiency for the worst case input of size n.
- The algorithm runs the longest among all possible inputs of that size.
- For the input of size n, the running time is $C_{worst}(n) = n$.

Best case efficiency

- The best-case efficiency of an algorithm is its efficiency for the best case input of size n.
- The algorithm runs the fastest among all possible inputs of that size n.
- In sequential search, If we search a first element in list of size n. (i.e. first element equal to a search key), then the running time is $C_{best}(n) = 1$

Average case efficiency

- The Average case efficiency lies between best case and worst case.
- To analyze the algorithm's average case efficiency, we must make some assumptions about possible inputs of size *n*.
- The standard assumptions are that
  - The probability of a successful search is equal to p $(0 \le p \le 1)$ and
  - The probability of the first match occurring in the ith position of the list is the same for every i.

$$C_{avg}(n) = [1 \cdot \frac{p}{n} + 2 \cdot \frac{p}{n} + \cdots + i \cdot \frac{p}{n} + \cdots + n \cdot \frac{p}{n}] + n \cdot (1 - p)$$
$$= \frac{p}{n}[1 + 2 + \cdots + i + \cdots + n] + n(1 - p)$$
$$= \frac{p}{n}\frac{n(n+1)}{2} + n(1 - p) = \frac{p(n+1)}{2} + n(1 - p).$$

SBC Karkala

ASYMPTOTIC NOTATIONS AND ITS PROPERTIES

Asymptotic notation is used to describe the efficiency of an algorithm based on how its running time grows with input size.

It is required to compare and rank the efficiency of algorithms.

The three asymptotic notations are: O – Big Oh , Ω – Big Omega , Θ – Big Theta

Let *t(n)* and *g(n)* can be any nonnegative functions defined on the set of natural numbers. The algorithm's running time *t(n)* usually indicated by its basic operation count *C(n),* and *g(n),* some simple function to compare with the count.

Example 1:

$n \in O(n^2)$,	$100n + 5 \in O(n^2)$,	$\frac{1}{2}n(n-1) \in O(n^2)$.
$n^3 \notin O(n^2)$,	$0.00001n^3 \notin O(n^2)$,	$n^4 + n + 1 \notin O(n^2)$.
$n^3 \in \Omega(n^2)$,	$\frac{1}{2}n(n-1) \in \Omega(n^2)$,	but $100n + 5 \notin \Omega(n^2)$.

where $g(n) = n^2$.

(i) O - Big oh notation

A function t*(n)* is said to be in *O(g(n))*, denoted $t\ (n) \in O(g(n))$, if *t (n)* is bounded above by some constant multiple of *g(n)* for all large *n,* i.e., if there exist some positive constant *c* and some nonnegative integer n_0 such that

$$t\ (n) \leq \text{ c . g } (n) \text{ for } all\ n\ \geq\ n_0.$$

Where *t(n)* and *g(n)* are nonnegative functions defined on the set of natural numbers.O = Asymptotic upper bound = Useful for worst case analysis

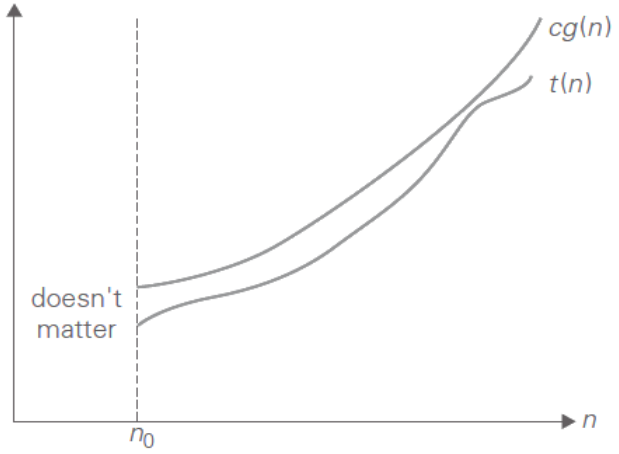


FIGURE 1.5 Big-oh notation: $t\ (n) \in$ O (g(n)).

Sri Karkala

Example 2: Prove the assertions $100n + 5 \in O(n^2)$.

Proof: $100n + 5 \leq 100n + n$ (for all $n \geq 5$)

$= 101n$

$\leq 101n^2$ ($\because n \leq n^2$)

Since, the definition gives us a lot of freedom in choosing specific values for constants **c** and $\mathbf{n_0}$. We have c=101 and n_0=5

Example 3: Prove the assertions $100n + 5 \in O(n)$.

Proof: $100n + 5 \leq 100n + 5n$ (for all $n \geq 1$)

$= 105n$

i.e., $100n + 5 \leq 105n$

i.e., $t(n) \leq cg(n)$

Hence $100n + 5 \in O(n)$ with c=105 and n_0=1

(ii) Ω - Big omega notation

A function *t(n)* is said to be in *Ω(g(n)),* denoted *t(n)* ∈ *Ω(g(n)),* if *t(n)* is bounded below by some positive constant multiple of *g(n)* for all large n, i.e., if there exist some positive constant c and some nonnegative integer n_0 such that

$t(n) \geq cg(n)$ for all $n \geq n_0$.

Where *t(n)* and *g(n)* are nonnegative functions defined on the set of natural numbers.

Ω = Asymptotic lower bound = Useful for best case analysis

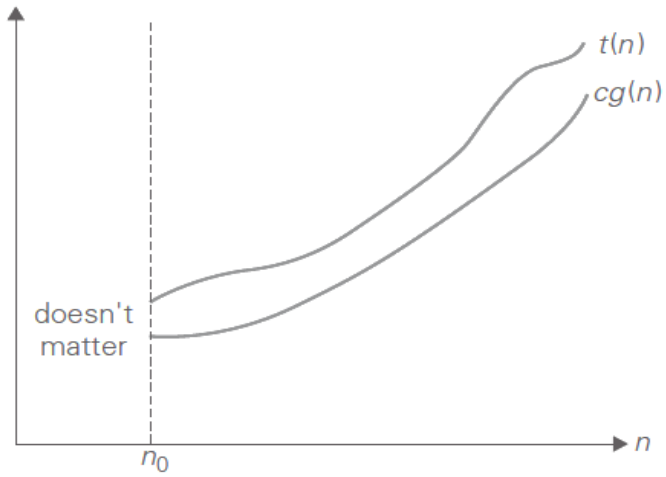


FIGURE 1.6 Big-omega notation: *t (n)* ∈ *Ω (g(n))*.

Example 4: Prove the assertions $n^3+10n^2+4n+2 \in \Omega(n^2)$.

Proof: $n^3+10n^2+4n+2 \geq n^2$ (for all $n \geq 0$)

i.e., by definition $t(n) \geq cg(n)$, where c=1 and n_0=0

(iii) Θ - Big theta notation

A function t(n) is said to be in *Θ(g(n)),* denoted t(n) ∈ Θ(g(n)), if t(n) is bounded both aboveand below by some positive constant multiples of g(n) for all large n, i.e., if there exist some positive constants c_1 and c_2 and some nonnegative integer n_0 such that

$c_2g(n) \leq t(n) \leq c_1g(n)$ for all $n \geq n_0$.

Where *t(n)* and *g(n)* are nonnegative functions defined on the set of natural numbers.

Θ = Asymptotic tight bound = Useful for average case analysis

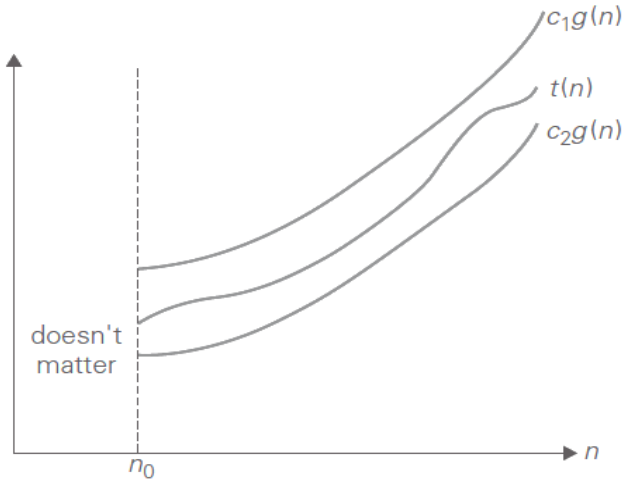


FIGURE 1.7 Big-theta notation: $t(n) \in \Theta(g(n))$.

Example 5: Prove the assertions $\frac{1}{2}n(n-1) \in \Theta(n^2)$.

Proof:

Upper bound:

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \leq \frac{1}{2}n^2.$$

Lower bound (for $n \geq 2$):

$$\frac{1}{2}n(n-1) = \frac{1}{2}n^2 - \frac{1}{2}n \geq \frac{1}{4}n^2.$$

Therefore,

$$\frac{1}{4}n^2 \leq \frac{1}{2}n(n-1) \leq \frac{1}{2}n^2.$$

Hence,

$$\boxed{\frac{1}{2}n(n-1) \in \Theta(n^2)}$$

where

$$\boxed{c_1 = \frac{1}{4},\ c_2 = \frac{1}{2},\ n_0 = 2.}$$

= ⇒ Θ() Exact bound

≤ ⇒ O() Upper bound

≥⇒ Ω() Lower bound

Basic rules of sum manipulation

$$\sum_{i=l}^{u} ca_i = c\sum_{i=l}^{u} a_i, \qquad \textbf{(R1)}$$

$$\sum_{i=l}^{u} (a_i \pm b_i) = \sum_{i=l}^{u} a_i \pm \sum_{i=l}^{u} b_i, \qquad \textbf{(R2)}$$

Summation formulas

$$\sum_{i=l}^{u} 1 = u - l + 1 \quad \text{where } l \le u \text{ are some lower and upper integer limits,} \qquad \textbf{(S1)}$$

$$\sum_{i=0}^{n} i = \sum_{i=1}^{n} i = 1 + 2 + \cdots + n = \frac{n(n+1)}{2} \approx \frac{1}{2}n^2 \in \Theta(n^2). \qquad \textbf{(S2)}$$

MATHEMATICAL ANALYSIS FOR RECURSIVE ALGORITHMS

General Plan for Analyzing the Time Efficiency of Recursive Algorithms

1. Decide on a *parameter* (or parameters) indicating an input's size.
2. Identify the algorithm's *basic operation*.
3. Check whether the *number of times the basic operation is executed* can vary on different inputs of the same size; if it can, the worst-case, average-case, and best-case *efficiencies* must be investigated separately.
4. *Set up a recurrence relation*, with an appropriate initial condition, for the number of times the basic operation is executed.
5. Solve the recurrence or, at least, ascertain the *order of growth* of its solution.

EXAMPLE 1: Compute the factorial function F(n) = n! for an arbitrary nonnegative integer n. Since $n! = 1 \cdot \ldots \cdot (n-1) \cdot n = (n-1)! \cdot n$, for n ≥ 1 and 0!= 1 by definition, we can compute $F(n) = F(n-1) \cdot n$ with the following recursive algorithm.

ALGORITHM *F(n)*

//Computes *n*! recursively
//Input: A nonnegative integer *n*
//Output: The value of *n*!
if *n* = 0 **return** 1
else return *F(n* − 1*)* * *n*

Algorithm analysis

- For simplicity, we consider *n* itself as an indicator of this algorithm's input size. i.e. 1.
- The basic operation of the algorithm is multiplication, whose number of executions we denote *M(n).* Since the function *F(n)* is computed according to the formula $F(n) = F(n-1) \cdot n$ for $n > 0$.
- The number of multiplications *M(n)* needed to compute it must satisfy the equality

$$M(n) = M(n-1) + 1 \quad \text{for } n > 0$$

M(n-1): To compute F(n-1); 1: To multiply F(n-1) by n

M(n − 1) multiplications are spent to compute *F(n − 1),* and one more multiplication is needed to multiply the result by *n.*

Recurrence relations

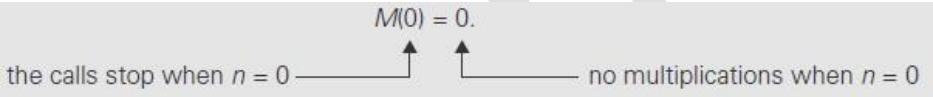
The last equation defines the sequence M(n) that we need to find. This equation defines M(n) not explicitly, i.e., as a function of n, but implicitly as a function of its value at another point, namely n − 1. Such equations are called *recurrence relations* or *recurrences*.

Solve the recurrence relation M(n) = M (n − 1) + 1, i.e., to find an explicit formula for *M(n)* in terms of *n* only.

To determine a solution uniquely, we need an initial condition that tells us the value with which the sequence starts. We can obtain this value by inspecting the condition that makes the algorithm stop its recursive calls:

if *n* = 0 **return** 1.

This tells us two things. First, since the calls stop when n = 0, the smallest value of n for which this algorithm is executed and hence M(n) defined is 0. Second, by inspecting the pseudocode's exiting line, we can see that when n = 0, the algorithm performs no multiplications.



Thus, the recurrence relation and initial condition for the algorithm's number of multiplications *M(n)*:

M(n) = M(n − 1) + 1 for n > 0,
M(0) = 0 for n = 0.

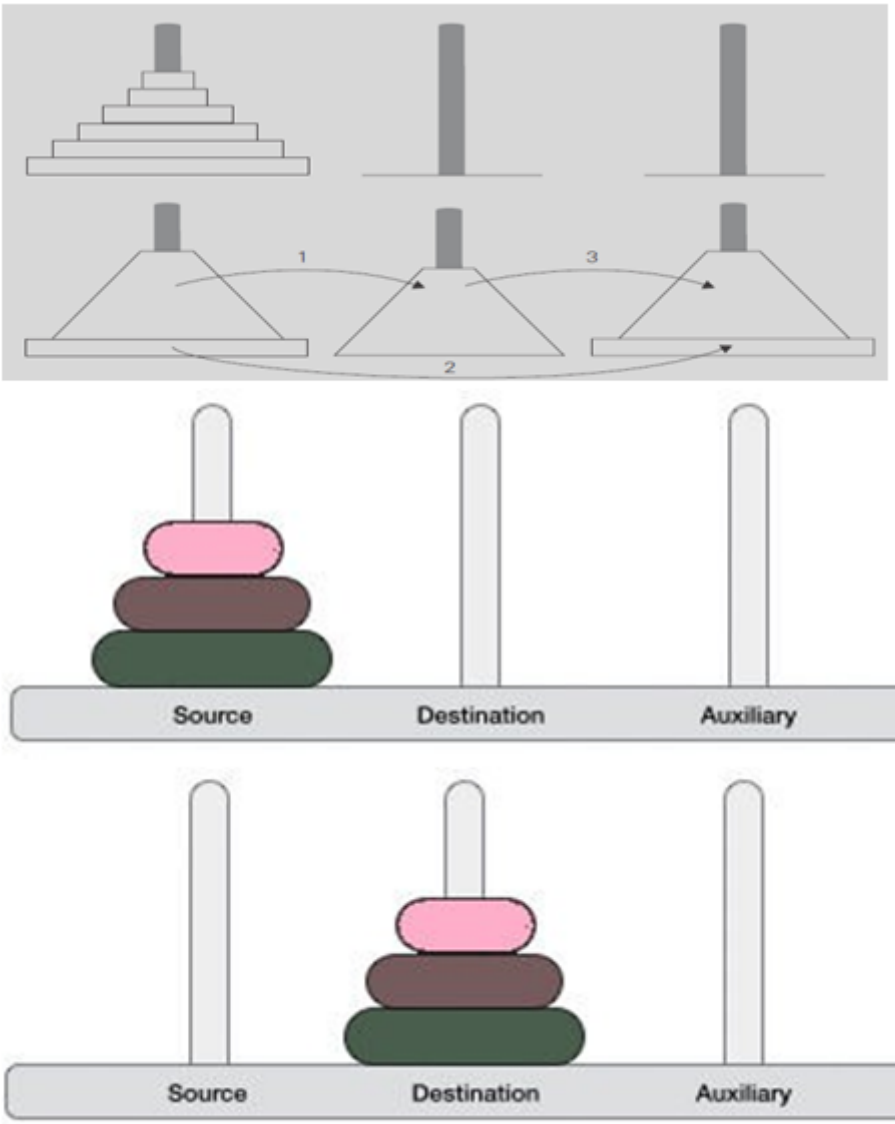
Method of backward substitutions

M(n) = *M(n − 1)* + 1 substitute *M(n − 1) = M(n − 2)* + 1
= [*M(n − 2)* + 1]+ 1
= *M(n − 2)* + 2 substitute *M(n − 2) = M(n − 3)* + 1
= [*M(n − 3)* + 1]+ 2
= *M(n − 3)* + 3
…
= M(n − i) + i
…
= M(n − n) + n
= n.

Therefore *M(n)=n*

EXAMPLE 2: consider educational workhorse of recursive algorithms: the ***Tower of Hanoi***puzzle. We have n disks of different sizes that can slide onto any of three pegs. Consider A (source), B (auxiliary), and C (Destination). Initially, all the disks are on the first peg in order of size, the largest on the bottom and the smallest on top. The goal is to move all the disks to the third peg, using the second one as an auxiliary.

SBC Karkala



Recursive solution to the Tower of Hanoi puzzle.

ALGORITHM TOH(n, A, C, B)

//Move disks from source to destination recursively

//Input: n disks and 3 pegs A, B, and C

//Output: Disks moved to destination as in the source order.

if n=1

    Move disk from A to C

else

    Move top n-1 disks from A to B using C

    TOH(n - 1, A, B, C)

    Move top n-1 disks from B to C using A

    TOH(n - 1, B, C, A)

Algorithm analysis

The number of moves $M(n)$ depends on n only, and we get the following recurrence equation for it: $M(n) = M(n-1) + 1 + M(n-1)$ for $n > 1$.

With the obvious initial condition $M(1) = 1$, we have the following recurrence relation for the number of moves $M(n)$:

$$M(n) = 2M(n-1) + 1 \text{ for } n > 1,$$
$$M(1) = 1.$$

We solve this recurrence by the same method of backward substitutions:

$M(n) = 2M(n-1) + 1$ sub. $M(n-1) = 2M(n-2) + 1$

$= 2[2M(n-2) + 1] + 1$

$= 2^2M(n-2) + 2 + 1$ sub. $M(n-2) = 2M(n-3) + 1$

$= 2^2[2M(n-3) + 1] + 2 + 1$

$= 2^3M(n-3) + 2^2 + 2 + 1$ sub. $M(n-3) = 2M(n-4) + 1$

$= 2^4M(n-4) + 2^3 + 2^2 + 2 + 1$

…

$= 2^iM(n-i) + 2^{i-1} + 2i^{-2} + \ldots + 2 + 1 = 2^iM(n-i) + 2^i - 1.$

…

Since the initial condition is specified for $n = 1$, which is achieved for $i = n - 1$,

$M(n) = 2^{n-1}M(n-(n-1)) + 2^{n-1} - 1 = 2^{n-1}M(1) + 2^{n-1} - 1 = 2^{n-1} + 2^{n-1} - 1 = 2^n - 1.$

Thus, we have an exponential time algorithm.

What is recursive relation? Give example

A **recurrence relation** is an equation that defines a problem in terms of **smaller instances of the same problem**.

Example for recurrence relation: Tower of Hanoi

$T(n)=2T(n-1)+1$ with the base condition: $T(1)=1$

Here, T(n) represents the number of moves required to solve the Tower of Hanoi problem with n disks.

MATHEMATICAL ANALYSIS FOR NON-RECURSIVE ALGORITHMS

General Plan for Analyzing the Time Efficiency of Nonrecursive Algorithms

1. Decide on a *parameter* (or parameters) indicating an input's size.
2. Identify the algorithm's *basic operation* (in the innermost loop).
3. Check whether the *number of times the basic operation is executed* depends only on the size of an input. If it also depends on some additional property, the worst-case, average-case, and, if necessary, best-case *efficiencies* have to be investigated separately.
4. Set up a *sum* expressing the number of times the algorithm's basic operation is executed.
5. Using standard formulas and rules of sum manipulation either find a closed form formula for the count or at the least, establish its *order of growth*.

SBCKarkala

EXAMPLE 1: Consider the problem of finding the value of **the largest element in a list of n numbers**. Assume that the list is implemented as an array for simplicity.

ALGORITHM MaxElement(A[0..n − 1])

```
//Determines the value of the largest element in a given array
//Input: An array A[0..n − 1] of real numbers
//Output: The value of the largest element in A
maxval ←A[0]
for i ←1 to n − 1 do
        if A[i]>maxval
                maxval←A[i]
return maxval
```

Algorithm analysis

- The measure of an input's size here is the number of elements in the array, i.e., n.
- There are two operations in the for loop's body:
  - The comparison A[i]> maxval and
  - The assignment maxval←A[i].
- The **comparison** is the basic operation because it is performed in every iteration of the loop.
- The number of comparisons is the same for every input of size **n**. Hence, **best, average, and worst cases are the same**.
- Let **C(n)** be the number of comparisons.
- The loop runs for **i = 1 to n − 1**, so the comparison is performed **n − 1 times**.
- Therefore,

$$C(n) = \sum_{i=1}^{n-1} 1$$

= n-1-1+1

$$\boxed{C(n) = n - 1 = \Theta(n)}$$

EXAMPLE 2: Consider the **element uniqueness problem**: check whether all the Elements in a given array of n elements are distinct.

ALGORITHM UniqueElements(A[0..n − 1])

```
//Determines whether all the elements in a given array are distinct
//Input: An array A[0..n − 1]
//Output: Returns "true" if all the elements in A are distinct and "false" otherwise
for i ←0 to n − 2 do
        for j ←i + 1 to n − 1 do
                if A[i]= A[j ] return false
return true
```

Algorithm analysis

- The input size is **n**, the number of elements in the array.
- The innermost loop performs one **comparison**, which is the basic operation.
- The number of comparisons depends on the arrangement of elements.
- Therefore, we consider the **worst case**.
- For each value of **i**, the inner loop runs from **j = i + 1 to n − 1**.
- The outer loop runs from **i = 0 to n − 2**.
- Hence, the total number of comparisons is:

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2} \sum_{j=i+1}^{n-1} 1$$

Evaluate the inner summation:

For a fixed i:

$$\sum_{j=i+1}^{n-1} 1 = n - 1 - (i+1) + 1 \quad = n - i - 1$$

Therefore,

$$C_{\text{worst}}(n) = \sum_{i=0}^{n-2} (n - i - 1)$$

$$= \sum_{i=0}^{n-2} (n-1) - \sum_{i=0}^{n-2} i \quad = (n-1) \sum_{i=0}^{n-2} 1 \; - \frac{(n-2)(n-1)}{2}$$

$$= \; (n-1) \cdot (n-1) - \frac{(n-2)(n-1)}{2}$$

$$= (n-1) \left[(n-1) - \frac{n-2}{2} \right] = (n-1) \left[\frac{2(n-1) - (n-2)}{2} \right]$$

$$= \; (n-1) \left[\frac{2n - 2 - n + 2}{2} \right] \quad = \; (n-1) \left[\frac{n}{2} \right]$$

$$= \frac{n^2 - n}{2} = \frac{1}{2} n^2 - \frac{1}{2} n \approx \frac{1}{2} n^2$$

Thus, the dominant term is n^2:

$$C_{\text{worst}}(n) \in \Theta(n^2)$$

Algorithm Sum(n)

S ← 0

for i ← 1 to n do

  S ← S + i

Return S

a) What does this algorithm compute?

It computes the **sum of the first n positive integers**.

$S=1+2+3+\cdots+n=\frac{n(n+1)}{2}$

b) What is its basic operation?

The **addition operation** S ← S + i is the basic operation.

c) How many times is the basic operation executed?

The loop runs from **1 to n**.

Therefore, the basic operation is executed **n times**.

d) What is the efficiency class?

Since the basic operation is executed n times: O(n)

---

2. Algorithm GUESS(A[][])

```
for i ← 0 to n-1
   for j ← 0 to i
      A[i][j] ← 0
```

a) What does the algorithm compute?

It assigns **0 to the lower triangular part of the matrix**, including the diagonal.

b) What is its basic operation?

The **assignment operation** A[i][j] ← 0 is the basic operation.

c) What is the efficiency of this algorithm?

For each i, the inner loop executes i+1 times.

Total executions: $1+2+3+\cdots+n=\frac{n(n+1)}{2}$

Therefore, the efficiency is: $O(n^2)$

Prove $100n + 5 = O(n^2)$

If a function $t(n) = O(g(n))$ then $t(n) \le c \cdot g(n)$ for all $n \ge n_0$

$t(n) = 100n + 5 \qquad g(n) = n^2$

For $n \ge 1 \qquad 5 \le 5n$

$\therefore \quad 100n + 5 \le 100n + 5n = 105n$

Since $n \ge 1 \qquad n \le n^2$

Multiply by 105

$105n \le 105n^2$

By combining $100n + 5 \le 105n^2$

$c = 105 \qquad n_0 = 1$

Hence $100n + 5 = O(n^2)$

Prove $5n^2 + 3n + 20 = O(n^2)$

If a function $t(n) = O(g(n))$ then $t(n) \le c \cdot g(n)$ for all $n \ge n_0$

$t(n) = 5n^2 + 3n + 20 \qquad g(n) = n^2$

For $n \ge 1 \qquad 3n \le 3n^2$ and $20 \le 20n^2$

$\therefore \quad 5n^2 + 3n + 20 \le 5n^2 + 3n^2 + 20n^2 = 28n^2$

$c = 28 \qquad n_0 = 1$

Hence $5n^2 + 3n + 20 = O(n^2)$

SBC Karkala

Prove $2n + 3 = \Omega(n)$

If a function $t(n) = \Omega(g(n))$ then $t(n) \geq c.g(n)$ for all $n \geq n_0$

$t(n) = 2n + 3 \qquad g(n) = n.$

We know that $3 \geq 0$

$\therefore \quad 2n + 3 \geq 2n$

$\therefore \quad c = 2 \qquad n_0 = 1$

Hence $2n + 3 = \Omega(n)$.

Prove $n^3 + 4n^2 = \Omega(n^2)$

If a function $t(n) = \Omega(g(n))$ then $t(n) \geq c.g(n)$ for all $n \geq n_0$

$t(n) = n^3 + 4n^2 \qquad g(n) = n^2$

We know that $n^3 \geq 0$

$\therefore \quad n^3 + 4n^2 \geq 4n^2$

$\therefore \quad c = 4 \qquad n_0 = 1$

Hence $n^3 + 4n^2 = \Omega(n^2)$.

Prove $n^2 + 5n + 7 = \Theta(n^2)$

If a function $t(n) = \Theta(g(n))$ then $c_1 g(n) \leq t(n) \leq c_2 g(n)$ for all $n \geq n_0$

$t(n) = n^2 + 5n + 7 \qquad g(n) = n^2$

For lower Bound $n^2 + 5n + 7 \geq n^2$

For upper bound $5n \leq 5n^2$ and $7 \leq 7n^2$

$\therefore \quad n^2 + 5n + 7 \leq n^2 + 5n^2 + 7n^2 = 13n^2$

$\therefore \quad n^2 \leq n^2 + 5n + 7 \leq 13n^2$

$c_1 = 1 \;,\; c_2 = 13 \qquad n_0 = 1$

Hence $n^2 + 5n + 7 = \Theta(n^2)$.

Prove $\frac{1}{2} n^2 + 3n = \Theta(n^2)$

If a function $t(n) = \Theta(g(n))$ then $c_1 g(n) \leq t(n) \leq c_2 g(n)$ for all $n \geq n_0$

$t(n) = \frac{1}{2} n^2 + 3n \qquad g(n) = n^2$

For lower bound $\frac{1}{2} n^2 + 3n \geq \frac{1}{2} n^2$

For upper bound For $n \geq 1$, $3n \leq 3n^2$

$\therefore \quad \frac{1}{2} n^2 + 3n \leq \frac{1}{2} n^2 + 3n^2 = \frac{7}{2} n^2$

$\frac{7}{2} < 4$, Hence $\frac{1}{2} n^2 \leq \frac{1}{2} n^2 + 3n \leq 4n^2$

$c_1 = \frac{1}{2} \;,\; c_2 = 4 \;,\; n_0 = 1$

Hence $\frac{1}{2} n^2 + 3n = \Theta(n^2)$.

SBC Karkala